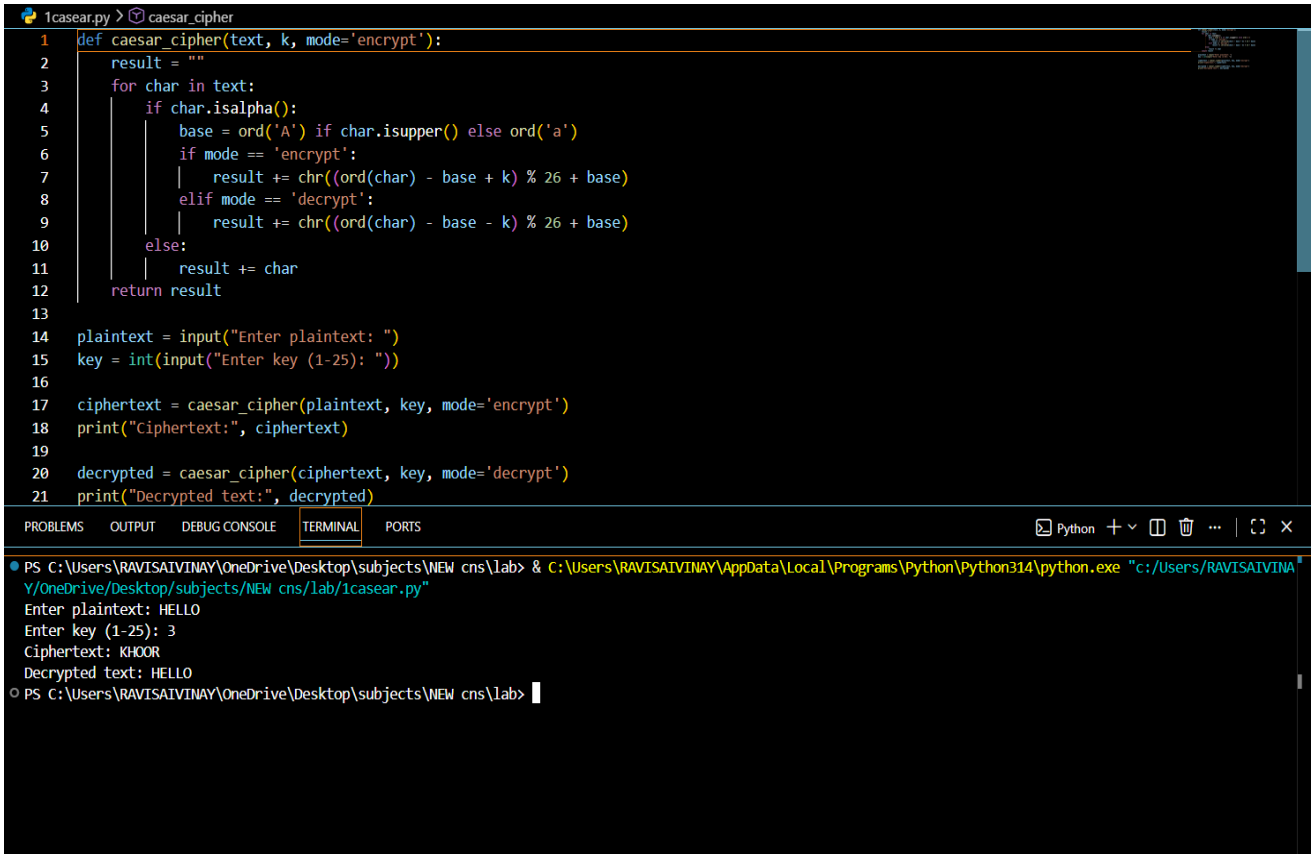
LIST OF PROGRAMS

1. Write a C program for Caesar cipher involves replacing each letter of the alphabet with the letter standing k places further down the alphabet, for k in the range 1 through 25.



```
1casear.py > caesar_cipher
def caesar_cipher(text, k, mode='encrypt'):
    result = ""
    for char in text:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')
            if mode == 'encrypt':
                result += chr((ord(char) - base + k) % 26 + base)
            elif mode == 'decrypt':
                result += chr((ord(char) - base - k) % 26 + base)
        else:
            result += char
    return result

plaintext = input("Enter plaintext: ")
key = int(input("Enter key (1-25): "))

ciphertext = caesar_cipher(plaintext, key, mode='encrypt')
print("Ciphertext:", ciphertext)

decrypted = caesar_cipher(ciphertext, key, mode='decrypt')
print("Decrypted text:", decrypted)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                Python

PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/1casear.py"
Enter plaintext: HELLO
Enter key (1-25): 3
Ciphertext: KHOOR
Decrypted text: HELLO
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

2. Write a C program for monoalphabetic substitution cipher maps a plaintext alphabet to a ciphertext alphabet, so that each letter of the plaintext alphabet maps to a single unique letter of the ciphertext alphabet.

2MonoAlpha.py > monoalphabetic_cipher

```python
def monoalphabetic_cipher(text, key_map):
    result = ""
    for char in text:
        if char.isalpha():
            base = 'A' if char.isupper() else 'a'
            idx = ord(char.upper()) - ord('A')
            mapped = key_map[idx]
            result += mapped.upper() if char.isupper() else mapped.lower()
        else:
            result += char
    return result

key_map = list("QWERTYUIOPASDFGHJKLZXCVBNM")  # substitution alphabet
plaintext = input("Enter plaintext: ")
ciphertext = monoalphabetic_cipher(plaintext, key_map)
print("Ciphertext:", ciphertext)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Programs\Python\Python314\python.exe "c:/User
Y/OneDrive/Desktop/subjects/NEW cns/lab/2MonoAlpha.py"
Enter plaintext: HELLO
Ciphertext: ITSSG
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

3. Write a C program for Playfair algorithm is based on the use of a 5 X 5 matrix of letters constructed using a keyword. Plaintext is encrypted two letters at a time using this matrix.

3Playfair.py > generate_matrix

```python
def generate_matrix(key):
    key = key.upper().replace("J", "I")
    matrix = []
    used = set()
    for c in key:
        if c.isalpha() and c not in used:
            matrix.append(c)
            used.add(c)
    for c in "ABCDEFGHIKLMNOPQRSTUVWXYZ":  # J merged with I
        if c not in used:
            matrix.append(c)
            used.add(c)
    return [matrix[i:i+5] for i in range(0, 25, 5)]

def print_matrix(matrix):
    for row in matrix:
        print(" ".join(row))

key = input("Enter keyword: ")
matrix = generate_matrix(key)
print("Playfair Matrix:")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Programs\Python\Python314\python.ex
Y/OneDrive/Desktop/subjects/NEW cns/lab/3Playfair.py"
Enter keyword: KEY
Playfair Matrix:
K E Y A B
C D F G H
I L M N O
P Q R S T
U V W X Z
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

4. Write a C program for polyalphabetic substitution cipher uses a separate monoalphabetic substitution cipher for each successive letter of plaintext, depending on a key.

```
4polyAlpha.py > ...
  1   def vigenere_cipher(text, key, mode='encrypt'):
 12           result += chr((ord(char) - ord(base) - shift) % 26 + ord(base))
 13           j += 1
 14       else:
 15           result += char
 16   return result
 17
 18   plaintext = input("Enter plaintext: ")
 19   key = input("Enter key (letters only): ")
 20
 21   ciphertext = vigenere_cipher(plaintext, key, 'encrypt')
 22   print("Ciphertext:", ciphertext)
 23
 24   decrypted = vigenere_cipher(ciphertext, key, 'decrypt')
 25   print("Decrypted:", decrypted)
 26

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Prog
rams\Python\Python314\python.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/4polyAlpha.p
y"
Enter plaintext: HELLO
Enter key (letters only): keu
Ciphertext: RIFVS
Decrypted: HELLO
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

5. Write a C program for generalization of the Caesar cipher, known as the affine Caesar cipher, has the following form: For each plaintext letter p, substitute the ciphertext letter C: C = E([a, b], p) = (ap + b) mod 26 A basic requirement of any encryption algorithm is that it be one-to-one. That is, if p q, then E(k, p) E(k, q). Otherwise, decryption is impossible, because more than one plaintext character maps into the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of a. For example, for a = 2 and b = 3, then E([a, b], 0) = E([a, b], 13) = 3.

a. Are there any limitations on the value of b?

b. Determine which values of a are not allowed.

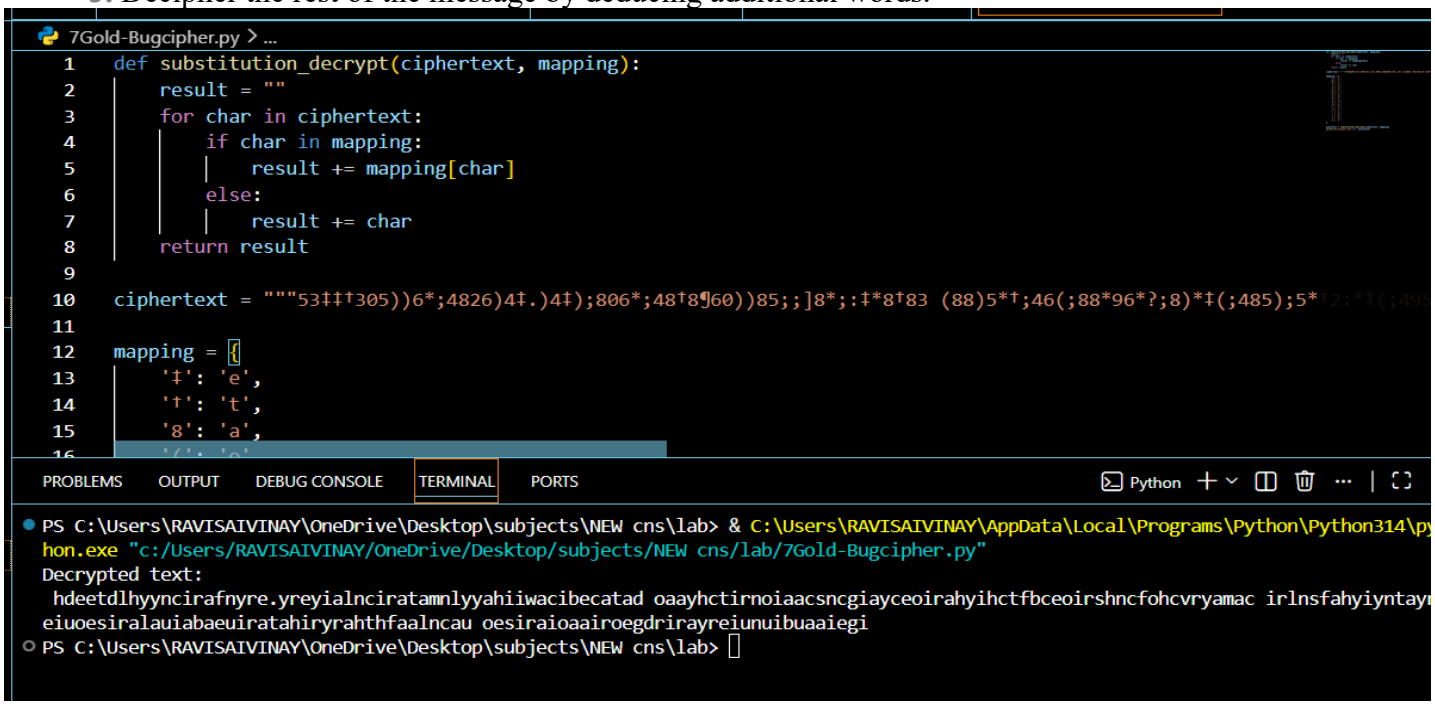
```
5affineCaesar.py > ...
  1   import math
  2
  3   def mod_inverse(a, m):
  4       a = a % m
  5       for x in range(1, m):
  6           if (a * x) % m == 1:
  7               return x
  8       return None
  9
 10   def affine_encrypt(text, a, b):
 11       if math.gcd(a, 26) != 1:
 12           raise ValueError("Invalid 'a'. Must be coprime with 26.")
 13       result = ""
 14       for char in text:
 15           if char.isalpha():
 16               base = 'A' if char.isupper() else 'a'
 17               result += chr(((a * (ord(char) - ord(base)) + b) % 26) + ord(base))
 18           else:
 19               result += char
 20       return result
 21

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Python

PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Programs\Python\Python3
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/5affineCaesar.py"
Valid values of 'a' are: {1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25}
Enter plaintext: HELLO
Enter a: 3
Enter b: 4
Ciphertext: ZQLLU
Decrypted: ZQLLU
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

6. Write a C program for ciphertext has been generated with an affine cipher. The most frequent letter of the ciphertext is “B,” and the second most frequent letter of the ciphertext is “U.”Break this code.

7. Write a C program for the following ciphertext was generated using a simple substitution algorithm.
53‡‡†305))6*;4826)4‡.)4‡);806*;48†8¶60))85;;]8*;:‡*8†83
(88)5*†;46(;88*96*?;8)*‡(;485);5*†2:*‡(;4956*2(5*—4)8¶8*
;4069285);)6†8)4‡‡;1(‡9;48081;8:8‡1;48†85;4)485†528806*81 (‡9;48;(88;4(‡?34;48)4‡;161;:188;‡?;

Decrypt this message.

1. As you know, the most frequently occurring letter in English is e. Therefore, the first or second (or perhaps third?) most common character in the message is likely to stand for e. Also, e is often seen in pairs (e.g., meet, fleet, speed, seen, been,
agree, etc.). Try to find a character in the ciphertext that decodes to e.
2. The most common word in English is “the.” Use this fact to guess the characters that stand for t and h.
3. Decipher the rest of the message by deducing additional words.



8. Write a C program for monoalphabetic cipher is that both sender and receiver must commit the permuted cipher sequence to memory. A common technique for avoiding this is to use a keyword from which the cipher sequence can be generated.

For example, using the keyword *CIPHER*, write out the keyword followed by unused letters in normal order and match this against the plaintext letters:

8ciperkey.py > generate_cipher_alphabet

```python
def generate_cipher_alphabet(keyword):
    keyword = keyword.upper()
    seen = set()
    cipher_alphabet = []
    for char in keyword:
        if char.isalpha() and char not in seen:
            cipher_alphabet.append(char)
            seen.add(char)
    for char in "ABCDEFGHIJKLMNOPQRSTUVWXYZ":
        if char not in seen:
            cipher_alphabet.append(char)
    return cipher_alphabet

def monoalphabetic_encrypt(plaintext, cipher_alphabet):
    plaintext = plaintext.upper()
    result = ""
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Progra
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/8ciperkey.py"
Cipher alphabet: CIPHERABDFGJKLMNOQSTUVWXYZ
Enter plaintext: HELLO
Ciphertext: BEJJM
Decrypted: HELLO
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

9. Write a C program for PT-109 American patrol boat, under the command of Lieutenant John F. Kennedy, was sunk by a Japanese destroyer, a message was received at an Australian wireless station in Playfair code:

KXJEY UREBE ZWEHE WRYTU HEYFS
KREHE GOYFI WTTTU OLKSY CAJPO
BOTEI ZONTX BYBNT GONEY CUZWR
GDSON SXBOU YWRHE BAAHY USEDQ

9Playfairkey.py > ...

```python
def playfair_decrypt(ciphertext, matrix):
                result += matrix[row1][col2]
                result += matrix[row2][col1]
    return result

keyword = "PLAYFAIR"
matrix = generate_matrix(keyword)

ciphertext = "KXJEY UREBE ZWEHE WRYTU HEYFS KREHE GOYFI WTTTU OLKSY CAJPO BOTEI ZONTX BYBNT GONEY CUZWR G
plaintext = playfair_decrypt(ciphertext, matrix)

print("Playfair Matrix:")
for row in matrix:
    print(row)
print("\nDecrypted Plaintext:\n", plaintext)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Programs\Python\Py
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/9Playfairkey.py"
Playfair Matrix:
['P', 'L', 'A', 'Y', 'F']
['I', 'R', 'B', 'C', 'D']
['E', 'G', 'H', 'K', 'M']
['N', 'O', 'Q', 'S', 'T']
['U', 'V', 'W', 'X', 'Z']

Decrypted Plaintext:
 CSPIPXIGIHXVMGHUCLNZGMAYKCIGGMRGAYBUSSNZGVCKXYPBLNRQNMDUNTSZCAIQOMNTKPIXXVLRCTNTKSRQXPVBGMAWWBPXNKBT
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

10. Write a C program for Playfair matrix:

M F H I/J K
U N O P Q
Z V W X Y
E L A R G
D S T B C

Encrypt this message: Must see you over Cadogan West. Coming at once.

10playfireMatrix.py > generate_matrix_from_list

```python
def generate_matrix_from_list(matrix_list):
    return [matrix_list[i:i+5] for i in range(0, 25, 5)]

def find_position(matrix, char):
    if char == 'J':
        char = 'I'
    for i in range(5):
        for j in range(5):
            if matrix[i][j] == char:
                return i, j
    return None

def preprocess_text(text):
    text = text.upper().replace("J", "I")
    text = "".join([c for c in text if c.isalpha()])
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Python

PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Progra
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/10playfireMatrix.py"
Playfair Matrix:
['M', 'F', 'H', 'I', 'K']
['U', 'N', 'O', 'P', 'Q']
['Z', 'V', 'W', 'X', 'Y']
['E', 'L', 'A', 'R', 'G']
['D', 'S', 'T', 'B', 'C']

Ciphertext:
 UZTBDLGZPNNWLGTGTUEROVLDBDUHFPERHWQSRZ
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

11. Write a C program for possible keys does the Playfair cipher have? Ignore the fact that some keys might produce identical encryption results. Express your answer as an approximate power of 2.

a. Now take into account the fact that some Playfair keys produce the same encryption results. How many effectively unique keys does the Playfair cipher have?

11.playfiredec.py > ...

```python
import math

letters = 25
total_keys = math.factorial(letters)
power_of_two_total = math.log2(total_keys)

unique_keys = 10**25
power_of_two_unique = math.log2(unique_keys)

print(f"{total_keys:.3e} ≈ 2^{power_of_two_total:.2f}")
print(f"{unique_keys:.3e} ≈ 2^{power_of_two_unique:.2f}")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Pyth

PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Pro
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/11.playfiredec.py"
1.551e+25 ≈ 2^83.68
1.000e+25 ≈ 2^83.05
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

12. a. Write a C program to Encrypt the message “meet me at the usual place at ten rather than eight oclock” using the Hill cipher with the key.

$$\begin{bmatrix} 9 & 4 \\ 5 & 7 \end{bmatrix}$$

a. Show your calculations and the result.
b. Show the calculations for the corresponding decryption of the ciphertext to recover the original plaintext.

12HCmeetme.py > mod_inv

```python
def mod_inv(a, m):
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

def text_to_nums(text):
    return [ord(c) - 97 for c in text if c != ' ']

def nums_to_text(nums):
    return ''.join(chr(n + 97) for n in nums)

def encrypt(pt, key):
    nums = text_to_nums(pt)
    if len(nums) % 2: nums.append(25)
    ct = []
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Programs\Python\Python.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/12HCmeetme.py"
Ciphertext: ukixukydromeiwszxwiokunukhxhroajroanqyebtlkjegir
Decrypted: meetmeattheusualplaceattenratherthaneightoclockz
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

13. Write a C program for Hill cipher succumbs to a known plaintext attack if sufficient plaintext–ciphertext pairs are provided. It is even easier to solve the Hill cipher if a chosen plaintext attack can be mounted.

13brokenhill.py > mod_inv

```python
def mod_inv(a, m):
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

def text_to_nums(text):
    return [ord(c) - 97 for c in text if c != ' ']

def nums_to_text(nums):
    return ''.join(chr(n + 97) for n in nums)

def encrypt(nums, key):
    ct = []
    for i in range(0, len(nums), 2):
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Prog
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/13brokenhill.py"
Plaintext samples: ['ba', 'ab']
Ciphertext samples: ['jf', 'eh']
Recovered key matrix:
[9, 4]
[5, 7]
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

14. Write a C program for one-time pad version of the Vigenère cipher. In this scheme, the key is a stream of random numbers between 0 and 26. For example, if the key is 3 19 5 . . . , then the first letter of plaintext is encrypted with a shift of 3 letters, the second with a shift of 19 letters, the third with a shift of 5 letters, and so on.

a. Encrypt the plaintext send more money with the key stream
9 0 1 7 23 15 21 14 11 11 2 8 9

b. Using the ciphertext produced in part (a), find a key so that the cipher text decrypts to the plaintext cash not needed.

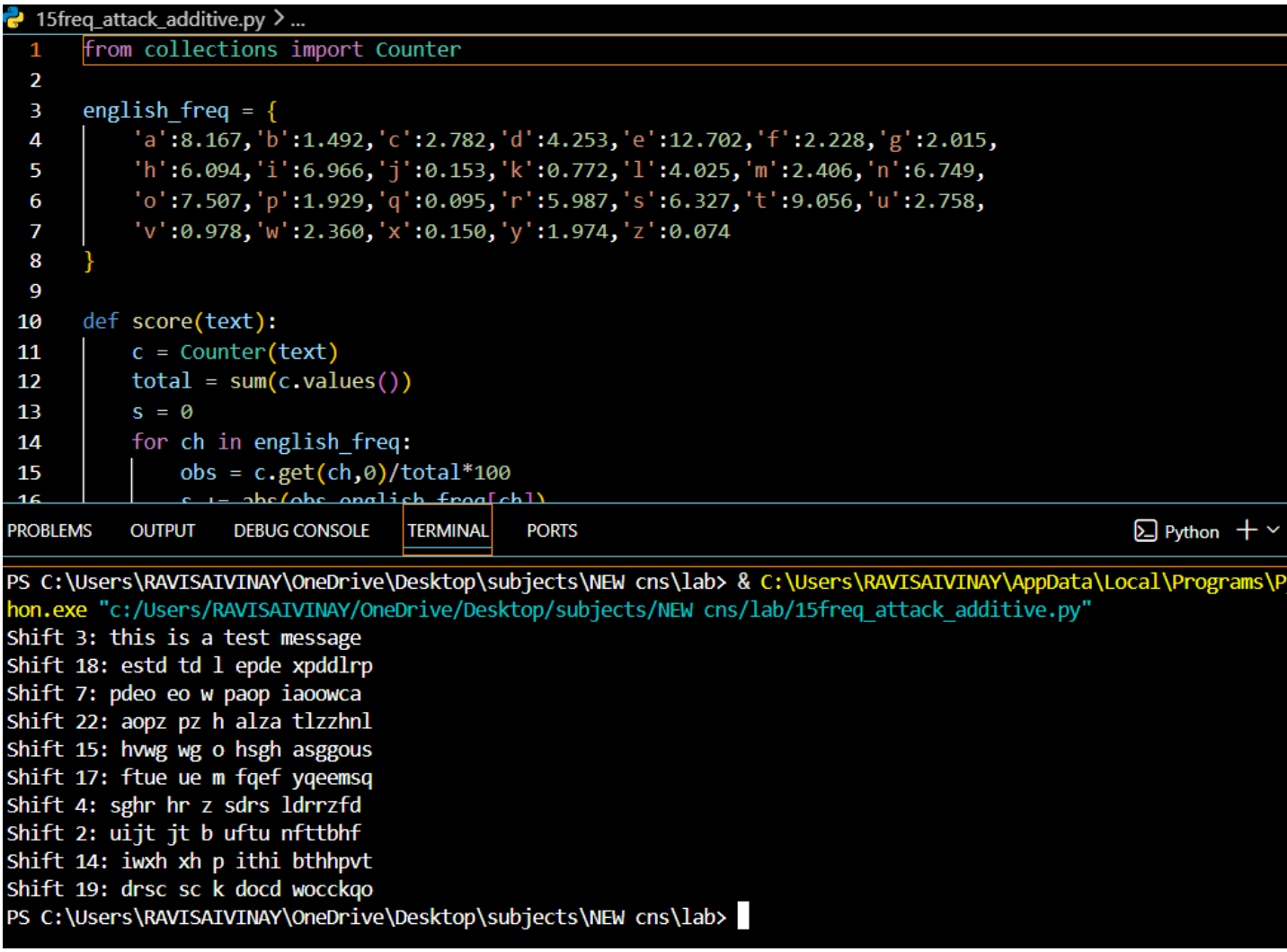
14otp_vigenere.py > text_to_nums

```python
def text_to_nums(t): return [ord(c)-97 for c in t if c.isalpha()]
def nums_to_text(n): return ''.join(chr(i+97) for i in n)

def encrypt(pt,key):
    n=text_to_nums(pt); c=[]
    for i in range(len(n)): c.append((n[i]+key[i])%26)
    return nums_to_text(c)

def decrypt(ct,key):
    n=text_to_nums(ct); p=[]
    for i in range(len(n)): p.append((n[i]-key[i])%26)
    return nums_to_text(p)

plaintext="sendmoremoney"
key=[9,0,1,7,23,15,21,14,11,11,2,8,9]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Loca
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/14otp_vigenere.py"
Ciphertext: beokjdmsxzpmh
New key: [25, 4, 22, 3, 22, 15, 19, 5, 19, 21, 12, 8, 4]
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

15. Write a C program that can perform a letter frequency attack on an additive cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be



16. Write a C program that can perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify "give me the top 10 possible plaintexts."

16freq_attack_substitution.py > ...

```python
from collections import Counter

english_freq_order = "etaoinshrdlcumwfgypbvkjxqz"

def text_to_nums(text):
    return [c for c in text.lower() if c.isalpha()]

def decrypt_with_mapping(cipher, mapping):
    return ''.join(mapping.get(c, c) for c in cipher.lower())

def frequency_attack(cipher, topn=10):
    letters = text_to_nums(cipher)
    freq = Counter(letters)
    sorted_cipher_letters = [item[0] for item in freq.most_common()]
    results = []
    for i in range(topn):
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Programs\P
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/16freq_attack_substitution.py"
Candidate 1: tnoe oe i taet saeeiha
Candidate 2: asit it n aota hottnro
Candidate 3: ohna na s oiao riaasdi
Candidate 4: irso so h inoi dnoohln
Candidate 5: ndhi hi r nsin lsiircs
Candidate 6: slrn rn d shns chnnduh
Candidate 7: hcds ds l hrsh ursslmr
Candidate 8: rulh lh c rdhr mdhhcwd
Candidate 9: dmcr cr u dlrd wlrrufl
Candidate 10: lwud ud m lcdl fcddmgc
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

17. Write a C program for DES algorithm for decryption, the 16 keys (K1, K2, c, K16) are used in reverse order. Design a key-generation scheme with the appropriate shift schedule for the decryption process.

17des_decrypt.py > SHIFT_SCHEDULE

```python
SHIFT_SCHEDULE = [1, 1, 2, 2, 2, 2, 2, 2,
                  1, 2, 2, 2, 2, 2, 2, 1]
def left_shift(bits, n):
    return bits[n:] + bits[:n]
def generate_round_keys(key56):
    C = key56[:28]
    D = key56[28:]
    round_keys = []
    for shift in SHIFT_SCHEDULE:
        C = left_shift(C, shift)
        D = left_shift(D, shift)
        round_keys.append(C + D)
    return round_keys
def feistel(right, subkey):
    return ''.join(str((int(r)^int(k))%2) for r,k in zip(right, subkey[:len(right)]))
def des_decrypt(ciphertext64, round_keys):
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/17des_decrypt.py"
Ciphertext: 1111000010101010111100001010101011110000101010101111000010101010
Plaintext (decrypted): 0110101000000011111010010010110010011111111110000100000100011110O
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

18. Write a C program for DES the first 24 bits of each subkey come from the same subset of 28 bits of the initial key and that the second 24 bits of each subkey come from a disjoint subset of 28 bits of the initial key.

```
18des_keygen.py > SHIFT_SCHEDULE
SHIFT_SCHEDULE = [1,1,2,2,2,2,2,2,
                  1,2,2,2,2,2,2,1]

def left_shift(bits, n):
    return bits[n:] + bits[:n]

def generate_subkeys(key56):
    C = key56[:28]
    D = key56[28:]
    subkeys = []
    for shift in SHIFT_SCHEDULE:
        C = left_shift(C, shift)
        D = left_shift(D, shift)
        # take first 24 bits from C and first 24 bits from D
        subkey = C[:24] + D[:24]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISA
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/18des_keyger
Round  1 subkey: 001001100011010001010111000110011011101111110011011
Round  2 subkey: 010011001101000101011101011001101111011100110111
Round  3 subkey: 001100110100010101110111110011011101111100111011111
Round  4 subkey: 110011010001010111011110001101110111110011011111111
Round  5 subkey: 001101000101011101111000110111011111001101111111111
Round  6 subkey: 110100010101110111100010011101111001101111111111100
Round  7 subkey: 010001010111011110001001110111100110111111111110010
Round  8 subkey: 000101011101110001001100111100110111111111001001
Round  9 subkey: 001010111011100010011001111001101111111110010011
Round 10 subkey: 101011101110001001100110100110111111111001001100
Round 11 subkey: 101110111000100110011010011011111111100100110011
Round 12 subkey: 111011100010011001101000101111111110010011001101
Round 13 subkey: 101110001001100110100010111111111001001100110111
Round 14 subkey: 111000100110011010001010111111100100110011011101
Round 15 subkey: 100010011001101000101011111110010011001101110111
Round 16 subkey: 000100110011010001010111111100100110011011101111
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

19. Write a C program for encryption in the cipher block chaining (CBC) mode using an algorithm stronger than DES. 3DES is a good candidate. Both of which follow from the definition of CBC.
Which of the two would you choose:

a. For security?
b. For performance?

```
19cbc.py > ...
from Crypto.Cipher import DES3
from Crypto.Random import get_random_bytes

key = DES3.adjust_key_parity(get_random_bytes(24))
iv = get_random_bytes(8)
cipher = DES3.new(key, DES3.MODE_CBC, iv)

plaintext = b"Secret Message 1234"
pad_len = 8 - (len(plaintext) % 8)
plaintext += bytes([pad_len]) * pad_len

ciphertext = cipher.encrypt(plaintext)
print("Ciphertext:", ciphertext)

decipher = DES3.new(key, DES3.MODE_CBC, iv)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Programs\Python\Py
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/19cbc.py"
Ciphertext: b'\xc2\xfcfI\xbf\x1e\xc4\xac;M\x93W\xd6\x9f\x13R\xc0\xc4\x87:C\xe1f|'
Decrypted: b'Secret Message 1234'
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

20. Write a C program for ECB mode, if there is an error in a block of the transmitted ciphertext, only the

corresponding plaintext block is affected. However, in the CBC mode, this error propagates. For example, an error in the transmitted C1 obviously corrupts P1 and P2.

a. Are any blocks beyond P2 affected?

b. Suppose that there is a bit error in the source version of P1. Through how many ciphertext blocks is this error propagated? What is the effect at the receiver?

20ecb.py > ...

```
3
4   k=get_random_bytes(16)
5   d=b'ABCDEFGHIJKLMNOP'
6
7   e=AES.new(k,AES.MODE_ECB)
8   c=e.encrypt(d)
9   print("ECB decrypt:",e.decrypt(c))
10
11  iv=get_random_bytes(16)
12  c=AES.new(k,AES.MODE_CBC,iv).encrypt(d)
13  print("CBC decrypt:",AES.new(k,AES.MODE_CBC,iv).decrypt(c))
14
15  # simulate error in C1 for CBC
16  c_err=bytearray(c);c_err[0]^=1
17  print("CBC with error:",AES.new(k,AES.MODE_CBC,iv).decrypt(bytes(c_err)))
18
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/20ecb.py"
ECB decrypt: b'ABCDEFGHIJKLMNOP'
CBC decrypt: b'ABCDEFGHIJKLMNOP'
CBC with error: b'\x07\xa0\xd8\xc7OE\xe4\xec3\x19p\xc5\xfc\xaa\xf0L'
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

21. Write a C program for ECB, CBC, and CFB modes, the plaintext must be a sequence of one or more complete data blocks (or, for CFB mode, data segments). In other words, for these three modes, the total number of bits in the plaintext must be a positive multiple of the block (or segment) size. One common method of padding, if needed, consists of a 1 bit followed by as few zero bits, possibly none, as are necessary to complete the final block. It is considered good practice for the sender to pad every message, including messages in which the final message block is already complete. What is the motivation for including a padding block when padding is not needed?

21ECB_CBC_CFB_Padding.py > ...

```
3   def pad(data, block_size):
4       pad_len = block_size - (len(data) % block_size)
5       return data + b'\x80' + b'\x00'*(pad_len-1)
6
7   key = b'1234567890123456'
8   iv = b'1234567890123456'
9   data = pad(b"HELLO WORLD",16)
10
11  cipher_ecb = AES.new(key,AES.MODE_ECB)
12  cipher_cbc = AES.new(key,AES.MODE_CBC,iv)
13  cipher_cfb = AES.new(key,AES.MODE_CFB,iv)
14
15  print(cipher_ecb.encrypt(data))
16  print(cipher_cbc.encrypt(data))
17  print(cipher_cfb.encrypt(data))
18
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Programs\Python\Python
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/21ECB_CBC_CFB_Padding.py"
b'\x8aV\xd7N\xebSl&<\x1a\xc3\xc3\xb8\xf9\x0f\xc8'
b"p\x9b-\x08\xb5G\x8c\x1c+O'\xc1\x0c\xee\xc9\x92"
b"=5np\xb5'\x8c\xc803~\xd9\xe3\xff\x88\x85"
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

22. Write a C program for Encrypt and decrypt in cipher block chaining mode using one of the following

ciphers: affine modulo 256, Hill modulo 256, S-DES, DES. Test data for S-DES using a binary initialization vector of 1010 1010. A binary plaintext of 0000 0001 0010 0011 encrypted with a binary key of 01111 11101 should give a binary plaintext of 1111 0100 0000 1011. Decryption should work correspondingly.

22CBC_SDES_Test.py > ...

```python
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

key = b'\x0f\xfd\x0f\xfd\x0f\xfd\x0f\xfd'  # 8-byte key
iv = b'\xaa'*8
plaintext = b'\x00\x01\x02\x03'

cipher = DES.new(key, DES.MODE_CBC, iv)
ciphertext = cipher.encrypt(pad(plaintext,8))
print("Ciphertext:", ciphertext)

decipher = DES.new(key, DES.MODE_CBC, iv)
decrypted = unpad(decipher.decrypt(ciphertext),8)
print("Decrypted:", decrypted)
```

OBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Progra
n.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/22CBC_SDES_Test.py"
phertext: b'\xc7t\xd4C\x87\xec\xcfx'
crypted: b'\x00\x01\x02\x03'
C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

23. Write a C program for Encrypt and decrypt in counter mode using one of the following ciphers: affine modulo 256, Hill modulo 256, S-DES. Test data for S-DES using a counter starting at 0000 0000. A binary plaintext of 0000 0001 0000 0010 0000 0100 encrypted with a binary key of 01111 11101 should give a binary plaintext of 0011 1000 0100 1111 0011 0010. Decryption should work correspondingly.

23_Counter_Mode_SDES.py > ...

```python
from Crypto.Cipher import DES
from Crypto.Util import Counter

key = b'\x0f\xfd\x0f\xfd\x0f\xfd\x0f\xfd'
ctr = Counter.new(64, initial_value=0)
plaintext = b'\x00\x01\x00\x02\x00\x04'

cipher = DES.new(key, DES.MODE_CTR, counter=ctr)
ciphertext = cipher.encrypt(plaintext)
print("Ciphertext:", ciphertext)

ctr = Counter.new(64, initial_value=0)
decipher = DES.new(key, DES.MODE_CTR, counter=ctr)
decrypted = decipher.decrypt(ciphertext)
print("Decrypted:", decrypted)
```

ROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
S C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Programs\P
on.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/23_Counter_Mode_SDES.py"
iphertext: b'n\xf4V\xfc%\xb9'
ecrypted: b'\x00\x01\x00\x02\x00\x04'
S C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

24. Write a C program for RSA system, the public key of a given user is e = 31, n = 3599. What is the

private key of this user? Hint: First use trial-and-error to determine p and q; then use the extended Euclidean algorithm to find the multiplicative inverse of 31 modulo f(n).

24_RSA_Private_Key.py > ...

```python
def egcd(a, b):
    if a == 0:
        return b, 0, 1
    g, y, x = egcd(b % a, a)
    return g, x - (b // a) * y, y

def modinv(a, m):
    g, x, y = egcd(a, m)
    if g != 1:
        raise Exception("No inverse")
    return x % m

n = 3599
e = 31

# Factor n
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\L
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/24_RSA_Private_Key.py"
p = 59
q = 61
phi = 3480
Private key d = 3031
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

25. Write a C program for set of blocks encoded with the RSA algorithm and we don't have the private key. Assume n = pq, e is the public key. Suppose also someone tells us they know one of the plaintext blocks has a common factor with n. Does this help us in any way?

25_RSA_Common_Factor.py > ...

```python

n = 77   # example modulus
block = 7  # example plaintext block

g = math.gcd(n, block)

print("n =", n)
print("block =", block)
print("gcd =", g)

if g > 1:
    print("This gcd reveals a factor of n:", g)
    print("RSA can be broken by factoring n using this.")
else:
    print("No useful factor found.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\Ap
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/25_RSA_Common_Factor.
n = 77
block = 7
gcd = 7
This gcd reveals a factor of n: 7
RSA can be broken by factoring n using this.
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

26. Write a C program for RSA public-key encryption scheme, each user has a public key, e, and a private key, d. Suppose Bob leaks his private key. Rather than generating a new modulus, he decides to generate a new public and a new private key. Is this safe?

26_RSA_KeyLeak.py > ...

```
3   def egcd(a, b):
5       return b, 0, 1
6       g, y, x = egcd(b % a, a)
7       return g, x - (b // a) * y, y
8
9   def modinv(a, m):
10      g, x, y = egcd(a, m)
11      if g != 1:
12          raise Exception("No inverse")
13      return x % m
14
15  # Example modulus
16  n = 3599
17  e = 31
18  factors = sympy.factorint(n)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\P
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/26_RSA_KeyLeak.py"
Leaked private key d = 3031
Factors of n: 59 61
Attacker recomputes new keys: 17 1433
Conclusion: Changing e,d with same n is unsafe. New modulus required.
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

27. Write a C program for Bob uses the RSA cryptosystem with a very large modulus n for which the factorization cannot be found in a reasonable amount of time. Suppose Alice sends a message to Bob by representing each alphabetic character as an integer between 0 and 25 (A S 0, c, Z S 25) and then encrypting each number separately using RSA with large e and large n. Is this method secure? If not, describe the most efficient attack against this encryption method.

27_RSA_SmallAlphabetAttack.py > ...

```
1   def rsa_encrypt(m, e, n):
2       return pow(m, e, n)
3
4   def rsa_decrypt(c, d, n):
5       return pow(c, d, n)
6
7   # Example: very large n (simulated with smaller for demo)
8   n = 3233   # pretend large modulus
9   e = 17
10  d = 2753
11
12  # Alice encodes letters A=0,...,Z=25
13  plaintext = "HELLO"
14  nums = [ord(ch)-65 for ch in plaintext]
15
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Program
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/27_RSA_SmallAlphabetAttack.py"
Ciphertext: [2369, 1387, 3061, 3061, 2549]
Decrypted: HELLO
Attack recovers: HELLO
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

28. Write a C program for Diffie-Hellman protocol, each participant selects a secret number x and sends the other participant ax mod q for some public number a. What would happen if the participants sent each other xa for some public number a instead? Give at least one method Alice and Bob could use to agree on a key. Can Eve break your system without finding the secret numbers? Can Eve find the secret numbers?

28_RSA_SmallAlphabetAttack_Improved.py > ...

```
1   def modexp(base, exp, mod):
7           exp = exp // 2
8           base = (base * base) % mod
9       return result
10
11  # Public parameters
12  q = 23
13  a = 5
14
15  # Alice and Bob choose secret numbers
16  xA = 6
17  xB = 15
18
19  # Normal Diffie-Hellman exchange
20  A = modexp(a, xA, q)
21  B = modexp(a, xB, q)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/28_RSA_SmallAlphabetAttack_Improved.py"
Alice sends: 8
Bob sends: 19
Shared key Alice: 2
Shared key Bob: 2
Alice sends wrong: 7
Bob sends wrong: 6
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

29. Write a C program for SHA-3 option with a block size of 1024 bits and assume that each of the lanes in the first message block (P0) has at least one nonzero bit. To start, all of the lanes in the internal state matrix that correspond to the capacity portion of the initial state are all zeros. Show how long it will take before all of these lanes have at least one nonzero bit. Note: Ignore the permutation. That is, keep track of the original zero lanes even after they have changed position in the matrix.

29_SHA3_BlockPropagation.py > ...

```
1   # Simulation of SHA-3 lane filling with 1024-bit block size
2   # We ignore permutation as instructed.
3
4   # SHA-3 state: 5x5 lanes, each lane is 64 bits
5   lanes = [[0 for _ in range(5)] for _ in range(5)]
6
7   # Capacity portion: assume half of the lanes (for 1024-bit capacity) start as zero
8   capacity_lanes = [(i,j) for i in range(5) for j in range(5)][12:]  # 13 lanes ~ 832 bits, close to 1024
9   for i,j in capacity_lanes:
10      lanes[i][j] = 0
11
12  # Message block P0: assume each lane in the rate portion has at least one nonzero bit
13  for i,j in [(i,j) for i in range(5) for j in range(5) if (i,j) not in capacity_lanes]:
14      lanes[i][j] = 1
15
16  # Track how many rounds until all capacity lanes get nonzero bits
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Programs\Python\Python\python.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/29_SHA3_BlockPropagation.py"
Rounds until all capacity lanes nonzero: 1
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

30. Write a C program for CBC MAC of a oneblock message X, say T = MAC(K, X), the adversary

immediately knows the CBC MAC for the two-block message X || (X ⊕ T) since this is once again.

30_CBC_MAC_Attack.py > ...

```python
def xor_bytes(b1, b2):
    return bytes([x ^ y for x, y in zip(b1, b2)])

def cbc_mac(key, message, block_size=16):
    # Simplified CBC-MAC: XOR with previous block, then "encrypt" by XOR with key
    prev = bytes([0]*block_size)
    for i in range(0, len(message), block_size):
        block = message[i:i+block_size]
        if len(block) < block_size:
            block = block + bytes([0]*(block_size-len(block)))
        prev = xor_bytes(xor_bytes(prev, block), key)
    return prev

# Example key and one-block message
key = b'\x01'*16
X = b'HELLOBLOCK12345'  # 16 bytes
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Progr
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/30_CBC_MAC_Attack.py"
MAC of X: b'IDMMNCMNBJ03254\x01'
MAC of X||(X⊕T): b'IDMMNCMNBJ03255\x01'
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

31. Write a C program for subkey generation in CMAC, it states that the block cipher is applied to the block that consists entirely of 0 bits. The first subkey is derived from the resulting string by a left shift of one bit and, conditionally, by XORing a constant that depends on the block size. The second subkey is derived in the same manner from the first subkey.

a. What constants are needed for block sizes of 64 and 128 bits?

b. How the left shift and XOR accomplishes the desired result.

31_CMAC_SubkeyGeneration.py > ...

```python
    return shifted.to_bytes(len(block), 'big')

def xor_bytes(b1, b2):
    return bytes([x ^ y for x, y in zip(b1, b2)])

def generate_subkeys(block_size_bits, L):
    if block_size_bits == 64:
        const = 0x1B
    elif block_size_bits == 128:
        const = 0x87
    else:
        raise ValueError("Unsupported block size")

    block_size = block_size_bits // 8
    L_bytes = L.to_bytes(block_size, 'big')
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\Appl
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/31_CMAC_SubkeyGenerati
K1: 2468acf121579bde2468acf121579bde
K2: 48d159e242af37bc48d159e242af37bc
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

32. Write a C program for DSA, because the value of k is generated for each signature, even if the same

message is signed twice on different occasions, the signatures will differ. This is not true of RSA signatures. Write a C program for implication of this difference?

32_DSA_RSA_SignatureDifference.py > ...

```
21  x = 3
22
23  msg = "HELLO"
24
25  # Two DSA signatures of same message
26  sig1 = dsa_sign(msg,p,q,g,x)
27  sig2 = dsa_sign(msg,p,q,g,x)
28  print("DSA signatures differ:", sig1, sig2)
29
30  # RSA demo
31  n = 3233
32  d = 2753
33  sig_rsa1 = rsa_sign(msg,n,d)
34  sig_rsa2 = rsa_sign(msg,n,d)
35  print("RSA signatures same:", sig_rsa1, sig_rsa2)
36
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Pytho

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Pro
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/32_DSA_RSA_SignatureDifference.py"
DSA signatures differ: (9, 6) (2, 10)
RSA signatures same: 2112 2112
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

33. Write a C program for Data encryption standard (DES) has been found vulnerable to very powerful attacks and therefore, the popularity of DES has been found slightly on the decline. DES is a block cipher and encrypts data in blocks of size of 64 bits each, which means 64 bits of plain text go as the input to DES, which produces 64 bits of ciphertext. The same algorithm and key are used for encryption and decryption, with minor differences. The key length is 56 bits. Implement in C programming.

33_DES_Implementation.py > ...

```
5   key = b'12345678'
6   cipher = DES.new(key, DES.MODE_ECB)
7
8   # Plaintext: 64 bits (8 bytes)
9   plaintext = b'ABCDEFGH'
10  print("Plaintext:", plaintext)
11
12  # Encrypt
13  ciphertext = cipher.encrypt(pad(plaintext, 8))
14  print("Ciphertext:", ciphertext)
15
16  # Decrypt
17  decipher = DES.new(key, DES.MODE_ECB)
18  decrypted = unpad(decipher.decrypt(ciphertext), 8)
19  print("Decrypted:", decrypted)
20
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppD
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/33_DES_Implementation.p
Plaintext: b'ABCDEFGH'
Ciphertext: b'\x96\xde`>\xae\xd6%o\xfe\xb9Y\xb7\xd4d/\xcb'
Decrypted: b'ABCDEFGH'
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

34. Write a C program for ECB, CBC, and CFB modes, the plaintext must be a sequence of one or more complete data blocks (or, for CFB mode, data segments). In other words, for these three modes, the total

number of bits in the plaintext must be a positive multiple of the block (or segment) size. One common method of padding, if needed, consists of a 1 bit followed by as few zero bits, possibly none, as are necessary to complete the final block. It is considered good practice for the sender to pad every message, including messages in which the final message block is already complete. What is the motivation for including a padding block when padding is not needed?

34_BlockCipherModes.py > ...

```
12
13    # ECB mode
14    cipher_ecb = AES.new(key, AES.MODE_ECB)
15    ct_ecb = cipher_ecb.encrypt(padded)
16    print("ECB ciphertext:", ct_ecb)
17
18    # CBC mode
19    cipher_cbc = AES.new(key, AES.MODE_CBC, iv)
20    ct_cbc = cipher_cbc.encrypt(padded)
21    print("CBC ciphertext:", ct_cbc)
22
23    # CFB mode
24    cipher_cfb = AES.new(key, AES.MODE_CFB, iv)
25    ct_cfb = cipher_cfb.encrypt(padded)
26    print("CFB ciphertext:", ct_cfb)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\A
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/34_BlockCipherModes.
ECB ciphertext: b'\x8aV\xd7N\xeb5l&<\x1a\xc3\xc3\xb8\xf9\x0f\xc8'
CBC ciphertext: b"p\x9b-\x08\xb5G\x8c\x1c+O'\xc1\x0c\xee\xc9\x92"
CFB ciphertext: b"=5np\xb5'\x8c\xc8O3~\xd9\xe3\xff\x88\x85"
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

35. Write a C program for one-time pad version of the Vigenère cipher. In this scheme, the key is a stream of random numbers between 0 and 26. For example, if the key is 3 19 5 . . . , then the first letter of plaintext is encrypted with a shift of 3 letters, the second with a shift of 19 letters, the third with a shift of

35_ C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab\34_BlockCipherModes.py

```
19    def decrypt(ciphertext, key):
27            plaintext.append(ch)
28        return "".join(plaintext)
29
30    # Example usage
31    plaintext = "HELLO"
32    key = generate_key(len(plaintext))
33    print("Plaintext:", plaintext)
34    print("Key:", key)
35
36    ciphertext = encrypt(plaintext, key)
37    print("Ciphertext:", ciphertext)
38
39    decrypted = decrypt(ciphertext, key)
40    print("Decrypted:", decrypted)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Lo
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/35_OneTimePad_Vigenere.py"
Plaintext: HELLO
Key: [6, 16, 12, 5, 13]
Ciphertext: NUXQB
Decrypted: HELLO
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

36. Write a C program for Caesar cipher, known as the affine Caesar cipher, has the following form: For each plaintext letter p, substitute the ciphertext letter C: C = E([a, b], p) = (ap + b) mod 26 A basic requirement of any encryption algorithm is that it be one-to-one. That is, if p q, then E(k, p) E(k, q). Otherwise, decryption is impossible, because more than one plaintext character maps into the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of a. For example, for a = 2 and b = 3, then E([a, b], 0) = E([a, b], 13) = 3.

```
36_AffineCaesarCipher.py > ...
35
36  # Example usage
37  plaintext = "HELLO"
38  a, b = 5, 8   # 'a' must be coprime with 26
39  ciphertext = affine_encrypt(plaintext, a, b)
40  print("Plaintext:", plaintext)
41  print("Ciphertext:", ciphertext)
42
43  decrypted = affine_decrypt(ciphertext, a, b)
44  print("Decrypted:", decrypted)
45
46  # Demonstrate non one-to-one case
47  a_bad, b_bad = 2, 3
48  print("E([2,3],0) =", (a_bad*0+b_bad)%26)
49  print("E([2,3],13) =", (a_bad*13+b_bad)%26)
50
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Lo
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/36_AffineCaesarCipher.py"
Plaintext: HELLO
Ciphertext: RCLLA
Decrypted: HELLO
E([2,3],0) = 3
E([2,3],13) = 3
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

37. Write a C program that can perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

```
37_FrequencyAttack_Monoalphabetic.py > ...
 7  def frequency_attack(ciphertext, top_n=10):
19              if i < len(english_freq_order):
20                  mapping[ch] = english_freq_order[(i+shift) % len(english_freq_
21          # Apply mapping
22          plaintext = "".join([mapping.get(ch, ch) for ch in ciphertext.upper()])
23          results.append(plaintext)
24      return results
25
26  # Example ciphertext (monoalphabetic substitution)
27  ciphertext = "WKH TXLFN EURZQ IRA MXPSV RYHU WKH ODCB GRJ"
28
29  print("Ciphertext:", ciphertext)
30  possible_plaintexts = frequency_attack(ciphertext, top_n=10)
31  for i, pt in enumerate(possible_plaintexts, 1):
32      print(f"Plaintext guess {i}:", pt)
33
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\App
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/37_FrequencyAttack_Mon
Ciphertext: WKH TXLFN EURZQ IRA MXPSV RYHU WKH ODCB GRJ
Plaintext guess 1: AOT SIHRD LNECU MEW FIGYP EBTN AOT VKJX QEZ
Plaintext guess 2: OIA HNRDL CSTUM WTF GNYPB TVAS OIA KJXQ ZTE
Plaintext guess 3: INO RSDLC UHAMW FAG YSPBV AKOH INO JXQZ EAT
Plaintext guess 4: NSI DHLCU MROWF GOY PHBVK OJIR NSI XQZE TOA
Plaintext guess 5: SHN LRCUM WDIFG YIP BRVKJ IXND SHN QZET AIO
Plaintext guess 6: HRS CDUMW FLNGY PNB VDKJX NQSL HRS ZETA ONI
Plaintext guess 7: RDH ULMWF GCSYP BSV KLJXQ SZHC RDH ETAO ISN
Plaintext guess 8: DLR MCWFG YUHPB VHK JCXQZ HERU DLR TAOI NHS
Plaintext guess 9: LCD WUFGY PMRBV KRJ XUQZE RTDM LCD AOIN SRH
Plaintext guess 10: CUL FMGYP BWDVK JDX QMZET DALW CUL OINS HDR
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

38. Write a C program for Hill cipher succumbs to a known plaintext attack if sufficient plaintext–

ciphertext pairs are provided. It is even easier to solve the Hill cipher if a chosen plaintext attack can be mounted. Implement in C programming.

38_HillCipher_KnownPlaintext.py > ...

```
2
3   def hill_encrypt(plaintext, key_matrix):
4       vec = np.array([ord(ch)-65 for ch in plaintext.upper()])
5       ct_vec = np.dot(key_matrix, vec) % 26
6       return "".join(chr(int(x)+65) for x in ct_vec)
7
8   # Example 2x2 Hill cipher
9   key = np.array([[3,3],[2,5]])
10  plaintext = "HI"
11  ciphertext = hill_encrypt(plaintext, key)
12  print("Plaintext:", plaintext)
13  print("Ciphertext:", ciphertext)
14
15  # Known plaintext attack: given enough (plaintext,ciphertext) pairs
16  # attacker can set up linear equations and solve for key matrix.
17
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAV
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/38_HillCi
Plaintext: HI
Ciphertext: TC
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

39. Write a C program that can perform a letter frequency attack on an additive cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

39_FrequencyAttack_Additive.py > ...

```
14  def frequency_attack(ciphertext, top_n=10):
15      counts = Counter([ch for ch in ciphertext.upper() if ch in string.ascii_uppercase])
16      most_common = counts.most_common(1)[0][0]
17      # Assume most common letter in ciphertext corresponds to 'E'
18      shift_guess = (ord(most_common)-65) - (ord('E')-65)
19      results = []
20      for delta in range(top_n):
21          shift = (shift_guess+delta) % 26
22          results.append(additive_decrypt(ciphertext, shift))
23      return results
24
25  ciphertext = "ZHOFRPH WR WKH FLSKHU"
26  print("Ciphertext:", ciphertext)
27  for i, guess in enumerate(frequency_attack(ciphertext, top_n=10),1):
28      print(f"Guess {i}:", guess)
29
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Programs
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/39_FrequencyAttack_Additive.py"
Ciphertext: ZHOFRPH WR WKH FLSKHU
Guess 1: WELCOME TO THE CIPHER
Guess 2: VDKBNLD SN SGD BHOGDQ
Guess 3: UCJAMKC RM RFC AGNFCP
Guess 4: TBIZLJB QL QEB ZFMEBO
Guess 5: SAHYKIA PK PDA YELDAN
Guess 6: RZGXJHZ OJ OCZ XDKCZM
Guess 7: QYFWIGY NI NBY WCJBYL
Guess 8: PXEVHFX MH MAX VBIAXK
Guess 9: OWDUGEW LG LZW UAHZWJ
Guess 10: NVCTFDV KF KYV TZGYVI
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```

40. Write a C program that can perform a letter frequency attack on any monoalphabetic substitution cipher

without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

40_FrequencyAttack_Monoalphabetic.py > ...

```python
import string
from collections import Counter

english_freq_order = "ETAOINSHRDLCUMWFGYPBVKJXQZ"

def frequency_attack(ciphertext, top_n=10):
    counts = Counter([ch for ch in ciphertext.upper() if ch in string.ascii_uppercase])
    cipher_freq_order = "".join([item[0] for item in counts.most_common()])
    results = []
    for shift in range(top_n):
        mapping = {}
        for i, ch in enumerate(cipher_freq_order):
            if i < len(english_freq_order):
                mapping[ch] = english_freq_order[(i+shift)%len(english_freq_order)]
        plaintext = "".join([mapping.get(ch,ch) for ch in ciphertext.upper()])
        results.append(plaintext)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab> & C:\Users\RAVISAIVINAY\AppData\Local\Program
hon.exe "c:/Users/RAVISAIVINAY/OneDrive/Desktop/subjects/NEW cns/lab/40_FrequencyAttack_Monoalphabetic.py"
Ciphertext: WKH TXLFN EURZQ IRA MXPSV RYHU WKH ODCB GRJ
Guess 1: AOT SIHRD LNECU MEW FIGYP EBTN AOT VKJX QEZ
Guess 2: OIA HNRDL CSTUM WTF GNYPB TVAS OIA KJXQ ZTE
Guess 3: INO RSDLC UHAMW FAG YSPBV AKOH INO JXQZ EAT
Guess 4: NSI DHLCU MROWF GOY PHBVK OJIR NSI XQZE TOA
Guess 5: SHN LRCUM WDIFG YIP BRVKJ IXND SHN QZET AIO
Guess 6: HRS CDUMW FLNGY PNB VDKJX NQSL HRS ZETA ONI
Guess 7: RDH ULMWF GCSYP BSV KLJXQ SZHC RDH ETAO ISN
Guess 8: DLR MCWFG YUHPB VHK JCXQZ HERU DLR TAOI NHS
Guess 9: LCD WUFGY PMRBV KRJ XUQZE RTDM LCD AOIN SRH
Guess 10: CUL FMGYP BWDVK JDX QMZET DALW CUL OINS HDR
PS C:\Users\RAVISAIVINAY\OneDrive\Desktop\subjects\NEW cns\lab>
```